



Project status report no.2

Team : SEA:ME

Prepared by : Minhyeok CHOI

Date : 02/02/2026

Reporting period : 22/12/2025 ~ 01/02/2026

Contents

1. Executive summary
2. Sprint goal & Key Deliverables
3. What we built & Current phase
4. Next step

1) Executive Summary

During this sprint, we developed a parking pipeline in simulation and established a reproducible workflow for safe, hardware-independent testing. The ROS 2–based system was validated through repeated simulation runs with stable, predictable parking behavior. In parallel, we enhanced our vision-based autonomy stack with dual-EKF fusion using ROS robot_localization, traffic sign–driven behavior planning, ROI-based obstacle avoidance, and adaptive-lookahead Pure Pursuit tracking. Next, we aim to integrate these modules and deploy a complete autonomous driving system on the real vehicle for the next qualification phase.

2) Sprint Goal & Key Deliverables

- ✓ Designed the PDC software architecture (planning and control scope)
- ✓ Set up a ROS 2–based BFMC simulation workflow for parking development
- ✓ Implemented parking local path generation and integrated it into the planning pipe
- ✓ Validated stable end-to-end parking maneuvers through repeated simulation runs
- ✓ Structured launch files for repeatable parking scenario testing
- ✓ Implemented global path generation and following pipeline
- ✓ Built a dual-EKF setup to improve vision-based localization stability
- ✓ Implemented traffic sign detection and sign-driven behavior planning
- ✓ Implemented obstacle vehicle detection and collision avoidance using dynamic ROI-based local path planning
- ✓ Stabilized path tracking with Pure Pursuit using an adaptive lookahead distance

3) What we built & Current phase

Localization System & Path pipeline (Repo : seame-simulation)

BEV-based lane features were matched to a lane map using a particle filter for vehicle localization. To improve stability when integrating GPS, we separated local and global estimation by introducing an odom→base_link frame for smooth relative motion and a map→odom frame for global correction. This reduced pose jumps and improved overall accuracy.

Global paths were generated from predefined graph edges with mandatory BFMC waypoints using the A* algorithm. Obstacle vehicles were handled via temporary lane-change local paths that rejoined the original route. Path tracking was stabilized using Pure Pursuit with a speed-adaptive lookahead distance.

Object Detection & Obstacle Avoidance (Repo: seame_simulation)

Object detection was implemented using a YOLO-based model trained on pre-collected data. Distance was estimated using a depth camera, enabling obstacle avoidance and traffic sign-based decision-making.

Global Path Generation & Following (Repo: seame_ros)

We generate the global path using the Hybrid A* algorithm and apply simple heuristic rules to improve path optimality and smoothness. To ensure consistent tracking, we align the path frame with the vehicle's base_link, derived from camera_link. We use VIO (visual-inertial odometry) for localization and global path following on the real vehicle.

Parking System & Development Pipeline (Repo: seame_simulation)

We implemented a parking-focused autonomy pipeline in simulation using coordinate-based local paths and low-speed planning/control logic. The pipeline is stable and fully validated through repeated simulation runs.

Current Phase

We are currently testing our stack on our lab track and in the BFMC simulation environment to improve robustness ahead of the March qualification.

4) Next step

In the next sprint, we will develop an obstacle-aware parking module that detects available parking spaces in dynamic scenes (e.g., vehicles and pedestrians) and autonomously tracks and parks in the target slot. After validating the pipeline in simulation, we will integrate it with localization and port it to the real vehicle for robustness and safety evaluation. We will also refine the control strategy to account for lane geometry by guiding the vehicle toward the lane centerline. Finally, we will install the delayed wheel encoder to complete the hardware setup.